Final Project: Weather Advisor Application

Course: ITMO 544 Cloud Computing / Cloud Security
Level: Graduate
Document type: Project overview and application architecture
Starting point: Your completed `lab06` (Python stack, DynamoDB, two-tier ALB and Auto Scaling design, `budget.py`, pytest tests). You will copy that folder to a new `final_project` folder and work only there.

---

1. Project Brief

You will design, build, deploy and test a **Weather Advisor** web application on the cloud infrastructure you have built through lab06.

A user fills in four fields:

Field	Rule
Date	A day within the **next 7 days**, counting from today
ZIP / postal code	A **five digit US ZIP code**. Postal codes from any other country are refused with a clear message (Section 5.4)
Gender	Used only to tailor the clothing suggestion
Email address	Where the report is sent if the user asks for it

The application then shows a report for that date and place, with **two columns, Morning and Night**, and these **ten** items in each column:

#	Item	Kind of item
1	Temperature	Measured value
2	Wind speed	Measured value
3	Visibility	Measured value
4	UV	Measured value
5	Humidity	Measured value
6	Rain (**Heavy / Mild / No**)	Category you derive from measured values
7	Snow (**Heavy / Mild / No**)	Category you derive from measured values
8	Suggested road conditions	Advice
9	Suggested dressing, based on gender	Advice
10	Suggested accessories to carry	Advice

A button on the report page sends the report to the email address the user typed.

What is fixed and what is yours to decide. The inputs, the outputs, the two data sources you must use (the **National Weather Service API** and **Google Gemini**), **Python for all deployment code**, and the deployment on your lab06 infrastructure are fixed. The language of the web application itself is yours to choose (Section 2). Everything else is a design decision that you make, document and defend: which other APIs you need, which language the application is written in, how the tiers are split, how you define "Morning", "Night", "Heavy" and "Mild", how email is sent, what is cached, and how the system behaves when something fails.

> 💡 **Note on the number of items.** The brief names ten items per period. Build and test all ten.

---

2. What You Reuse from Lab06 and What Is New

Area	Carries over from lab06	New in this project
Infrastructure	Two ALB listeners, two Auto Scaling Groups, restricted app security group, per-tier IAM roles, DynamoDB tables with TTL and no backups	A different set of tiers if your design needs it (Section 4)
Deployment code	The Python `stack/` package, idempotent `ensure_*` and delete functions, the Python boot script, `create_` and `delete_` entry scripts	New settings, new IAM permissions, new tables or keys
Weather data	The National Weather Service lookup, zip code lookup and caching idea from the weather service	Date selection, Morning and Night periods, more fields, a source for every field
AI	Calling a model, a call cap, a fallback when the model is unavailable, saved answers	**Gemini instead of Bedrock**; the key lives in SSM Parameter Store; structured answers; three kinds of advice instead of one
Application code	The two Node.js (Express) apps: a weather service and an advisor service	You may **extend the lab06 Node.js apps** or **write the application in Python**. Either is acceptable. Record the choice and the reason (Section 12)
Cost control	`budget.py`, a daily call cap, short answers	Gemini free-tier limits instead of Bedrock prices
New feature	none	Email delivery of the report (Section 8)

There is no bash anywhere in this project. The deployment code is **Python**, as in lab06: the boto3 stack, the boot script that starts the application on each instance, the create and delete scripts, `budget.py`, and the infrastructure and black-box tests. The web application itself may be Node.js or Python.

---

3. Learning Objectives

By the end of this project you will be able to:

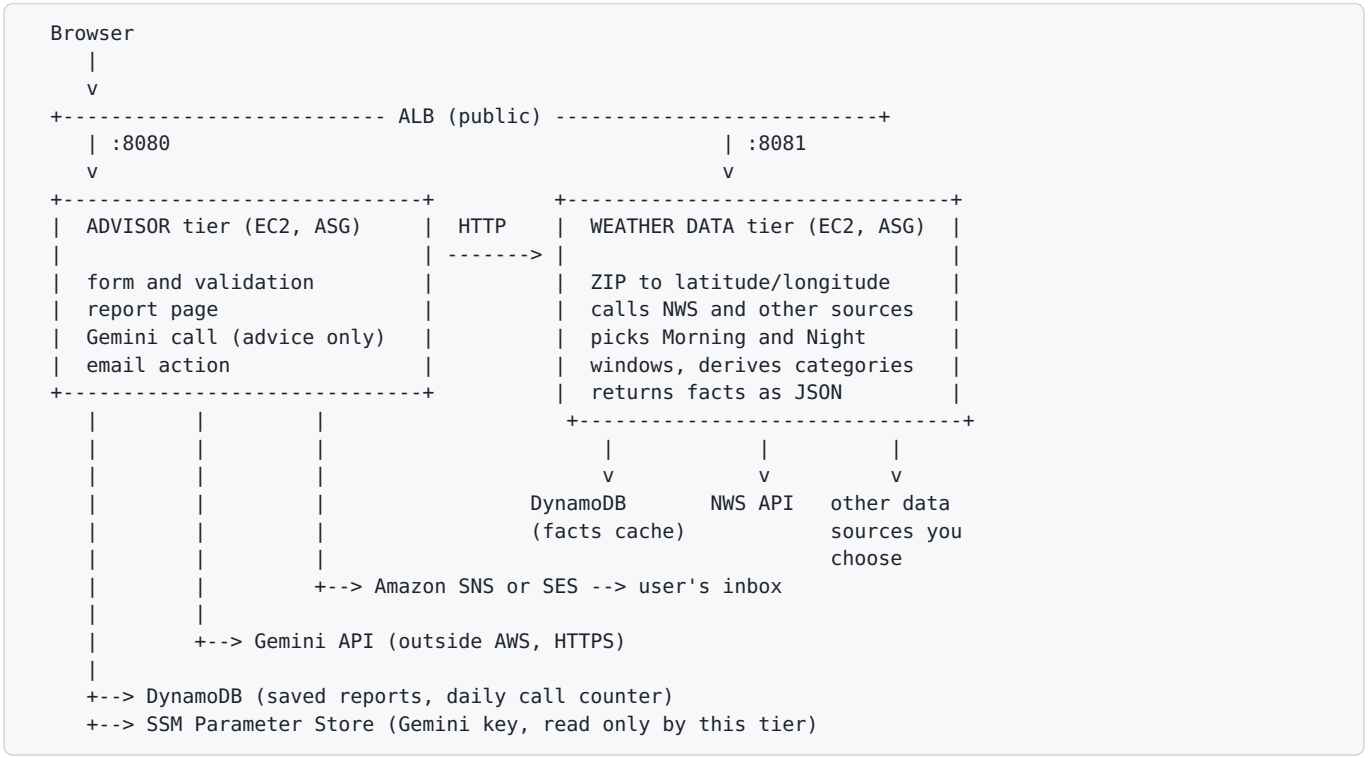
1. Turn a short product brief into a **cloud application architecture**, with a diagram, component responsibilities, data flows and a recorded reason for each decision.
2. **Evaluate public APIs** against a list of required outputs, find the gaps, and choose and justify additional sources.
3. Separate **facts that must come from data** from **advice that may come from a language model**, and enforce the separation in code.
4. Design for **failure of every outside dependency** and show what the user sees when each one fails.
5. Apply **least privilege**, secret handling, input validation and abuse controls to a public web application.
6. Choose between AWS email services and explain the trade-offs (SNS and SES).
7. Deploy and tear down the application with **idempotent Python infrastructure code** (no bash, including the boot script) and prove it with moto, integration and chaos tests.
8. Estimate and report the **cost** of what you built.

---

4. Reference Architecture

This is the architecture we expect, and the starting point for yours. You may change it, but every change must be

written up as a decision (Section 12).



4.1 Components and responsibilities

Component	Responsibility	Must not do
ALB	Public entry point, health checks, forwards to the two tiers	Expose the app instances directly (the app security group accepts the app port from the ALB only)
Advisor tier	Serve the form, validate input, ask the data tier for facts, ask Gemini for advice, render the report, handle the email request	Talk to the weather APIs itself, or hold weather credentials
Weather data tier	Turn a ZIP and a date into **normalized facts** for Morning and Night, from the sources you choose, with caching	Call Gemini, send email, or know anything about the user's gender or address
DynamoDB	Cache of facts (by place and date), saved reports, the daily Gemini call counter. On-demand, TTL on every item, no backups, as in lab06	Keep personal data longer than needed
SSM Parameter Store	Holds the Gemini API key as an encrypted value	Be readable by any role except the advisor's
Gemini API	Writes the three advice items from the facts you give it	Supply any measured number
Email service (SNS or SES)	Delivers the report to the user's address	Deliver one person's report to another person
Python stack code	Creates and deletes every AWS resource, idempotently	Contain a shell command or a hand-run step

You may merge the two tiers into one or add a third. A merge is acceptable only if you can explain what you lose (independent scaling, a smaller blast radius, separate permissions). A split that has no reason is also a weakness.

4.2 Request flow: building a report

Step	What happens	Where
1	The user submits date, ZIP, gender and email	Browser to advisor tier
2	The advisor tier **validates** every field (Section 7.1). A bad field ends the request with a clear message	Advisor tier
3	If a saved report for the same place, date and gender exists and is still fresh, it is shown at once	Advisor tier, DynamoDB
4	Otherwise the advisor tier asks the data tier for the facts for that ZIP and date	Advisor tier to data tier
5	The data tier finds the location, gets the forecast from the weather sources, picks the Morning and Night periods, computes the ten facts and the Rain and Snow categories, and returns them. It caches the result	Data tier, outside APIs, DynamoDB
6	The advisor tier checks the **daily Gemini call cap**, then sends Gemini the facts (not the email, not the ZIP's owner, nothing personal) and the clothing category for the chosen gender	Advisor tier to Gemini
7	The advisor tier **checks the answer** (shape, lengths, allowed values) and merges the advice with the facts	Advisor tier
8	The report is saved briefly and shown, with each fact labeled with where it came from	Advisor tier

4.3 Request flow: emailing a report

Step	What happens
1	The user clicks the email button on the report page
2	The advisor tier checks that the address is valid and that this visitor has not exceeded the email limit
3	The advisor tier makes sure the address has **agreed to receive mail** (Section 8). If not, it starts that process and tells the user what to do next, without sending the report
4	When the address is ready, the report is sent, only to that address
5	The user sees a clear status: sent, waiting for confirmation, or failed

The email address is used for this one purpose. It is never sent to Gemini and never written to a log.

4.4 Where each kind of data comes from

This is the most important rule of the design:

Kind	Examples	Allowed source	Why
Measured or forecast values	Temperature, wind speed, visibility, UV, humidity, rain and snow amounts	**Weather data APIs only**	A language model can invent a plausible number. A weather report must not
Derived categories	Rain: Heavy, Mild or No. Snow: Heavy, Mild or No	**Your code**, using thresholds you document	The category must be the same every time for the same data, and testable
Advice	Road conditions, dressing, accessories	**Gemini**, given the facts	This is where a model adds value: turning numbers into practical suggestions

Gemini receives the facts and must not change them. The code then checks its answer, and the page shows the facts from the data tier, not facts repeated back by the model.

---

5. Choosing the Data Sources

"Identify the APIs that can provide all the output" is a deliverable (Section 13), not a hint. The method is yours. Use this process.

5.1 Build a source map

For **each of the ten outputs and for each of the two periods**, record:

Column	Question
Output	Which of the ten?
Source	Which API, and which endpoint and field?
Evidence	The exact field name, unit and a real example value you retrieved, not something you assumed
Horizon	Does it cover the **whole 7 day window**? How far ahead does it really go?
Resolution	Hourly, every few hours, or daily?
Gap handling	What does the report show when the field is missing?

5.2 Questions your investigation must answer

- What does the **National Weather Service API** actually return for a point, as a **text forecast** and as **raw gridded data**, and which of your ten outputs does each cover? Do not assume. Retrieve real responses for a real ZIP and read every field name.
- Does the NWS API provide **every** output for **both** periods on **every** day up to day 7? Where it does not, what is the gap?
- For every gap, find a **second source** and justify it, or decide that the value is **derived** or **explicitly unavailable**. A report that quietly shows zero for a missing value is a failure.
- How do you turn a **ZIP code** into the coordinates the weather API needs, and what does that service return for a ZIP that does not exist?

5.3 Evaluation criteria for a data source

Criterion	Why it matters
Covers the field **and** the 7 day horizon	A source that stops at day 3 cannot serve the project
Free to use, and whether it needs a key	Your budget is small, and keys must be stored safely
Rate limits and fair use rules	The service must keep working when several people use it
Terms of use and required attribution	You must follow them and show attribution in the report
Reliability and how it fails	Your failure behavior depends on it
Units and time zones	Mixing them is the commonest source of wrong numbers

5.4 Scope rules you must state

- **Coverage.** The National Weather Service covers the **United States and its territories**, so this application supports **five digit US ZIP codes only**. Any other postal code (for example a Canadian code such as `M5V 3L9`, a UK code such as `SW1A 1AA`, or a number with the wrong number of digits) must be **refused with a clear message** that says the service covers US locations only and what a valid entry looks like. The refusal happens on the server **before any outside call** (no geocoding request, no weather request, no Gemini call), and it never repeats the user's input back without escaping it. A five digit code that matches the pattern but does not exist is a different case and gets its own "ZIP code not found" message.

- **Time.** "Morning", "Night" and "the selected date" must be in the **local time of the ZIP code's location**, not the server's time and not UTC.
- **Freshness.** Forecasts change. State how long your cached facts stay valid and why.

---

6. Defining the Report

You must write down, in your architecture document, a **definition for every row** of the report. The definitions below are the ones you must settle. Examples are suggestions, and you may choose differently if you can explain why.

6.1 Periods

Decision	Suggested starting point
Morning	06:00 to 11:59 local time on the selected date
Night	18:00 to 23:59 local time on the selected date (so the period does not cross midnight)
Date range	Today through today plus 6 days (seven calendar days). State whether "today" can be chosen late in the evening, when Morning has already passed

6.2 Aggregation: one number per period

A period covers several hours of forecast. For each measured item, decide **which single value or short range** you show and why.

Item	Decision you must make
Temperature	Low and high, an average, or the value at the middle of the period? Does the "feels like" value matter for the advice?
Wind speed	Sustained wind, gusts, or both? Which unit?
Visibility	Lowest value in the period or the average? Which unit? What counts as poor visibility?
UV	Peak value or average? Which scale does the source use? What does a UV value mean for a user at night?
Humidity	Average or range?
Rain	Which measure feeds the category: the **chance** of precipitation, the forecast **amount**, or both?
Snow	The same questions, with snowfall amount

6.3 Heavy, Mild and No

You must publish **thresholds** for the Rain and Snow categories, using the units of your source, and cite where the thresholds come from (for example a published convention for rain rates used in meteorology). Explain how the chance of precipitation is combined with the amount, and what happens when only one of them is available. Test the boundaries (a value exactly on a threshold, just below and just above).

6.4 The three advice items

Item	What a good answer looks like	Inputs it may use
Suggested road conditions	A short, practical statement (for example about wet or icy roads, reduced visibility, strong crosswinds)	Rain, snow, temperature, visibility, wind
Suggested dressing, based on gender	Clothing suited to the temperature, wind and precipitation, tailored to the gender category the user chose	Temperature, wind, humidity, rain, snow, gender category
Suggested accessories to carry	Items such as an umbrella, sunscreen, sunglasses, gloves or a scarf	Rain, snow, UV, temperature, wind

Decide how you handle **gender** responsibly:

- Offer a short, inclusive list of choices that includes a neutral option, and say in the interface that it is used only to tailor clothing suggestions.
- Give Gemini a **clothing category** (for example womenswear, menswear or neutral), not an identity or a free-text field.
- Do not store gender with the email address, and do not log either.

---

7. Security and Safety Requirements

7.1 Validation (every field, on the server)

Field	What must be enforced
Date	A real calendar date, inside the 7 day window. Reject past dates, far future dates and impossible dates
ZIP / postal code	Exactly five digits. Reject every other format, including other countries' postal codes, before any outside call is made. Treat a well formed ZIP that does not exist separately
Gender	One of a fixed list. Never free text
Email	A valid address of limited length. Normalized the same way every time (for example lowercase)

Browser checks are only a convenience. The server decides.

7.2 Secrets and permissions

- The Gemini key lives **only** in SSM Parameter Store (encrypted). It is not in code, not in `.env`, not in user-data, not in a log, not in a screenshot.
- **Each tier has its own IAM role** with only the actions it uses, on only the resources it uses. Document each permission with the line of code that needs it. No wildcard actions, no wildcard resources.
- The app security group accepts the app port **only from the ALB's security group**.

7.3 Abuse controls

A public page that calls a paid or limited API, and can send email, will be abused. You must have:

Risk	Control you must design
One visitor exhausting the Gemini free tier	A per-visitor rate limit and a **daily cap** on Gemini calls, with a fallback when the cap is reached
The page used to send mail to strangers	Email only after the recipient has confirmed (Section 8), plus a per-visitor limit on email requests
Repeated identical requests	Saved reports and cached facts, so the same question is not answered twice
Hostile text in any field or API response	The model's answer and every API response are **checked and escaped** before display. Nothing from outside is trusted
Prompt injection	Only fixed, validated values reach the model. No free text from the user is sent to it

7.4 Privacy

Collect the least you need. Do not keep the email address or gender longer than the request needs them, other than what the email service itself must hold to deliver mail. Items in DynamoDB have an expiry. Say in the interface what is sent to Google (the weather facts and a clothing category) and what is not.

---

8. Email Delivery: A Required Architecture Decision

The report must be emailed to the address the user entered. AWS offers two services you could use. **Choose one, and document why.**

	Amazon SNS	Amazon SES
What it is	A publish and subscribe service. Email is one delivery option	A full email sending service
Message format	**Plain text only**	Plain text and HTML, and attachments
Who may receive mail	Each recipient must **confirm a subscription** by clicking a link	A new account is in a **sandbox**: both the sender and every recipient address must be verified, until you request production access
Keeping one person's mail away from another	A shared topic sends to every subscriber unless you design **filtering** (subscription filter policies and message attributes) or use a topic per recipient	Each message goes to the address you name
Setup effort	Low	Higher (sender identity, verification, sandbox)
Cost at this scale	Free tier covers it	Very small
Good fit when	Plain text is enough	You want a formatted report

Whichever you choose, your design must answer:

1. How does the recipient give **consent**, and what does the user see while they wait for it?
2. How do you make sure the report goes **only to the person who asked**?
3. What does the email contain? (A report is text and tables, so decide how a table reads in plain text, or how HTML is built safely.)
4. What happens when the confirmation expires, the address is wrong, or the service is unavailable?
5. What permissions does the advisor role need, and on which resource?
6. How do you prove in a test that two different recipients never see each other's report?

---

9. Reliability and Failure Behavior

Every outside dependency will fail sometimes. For each row, your design must say what the user sees and what the system does. "An error page" is not an answer.

Failure	Required behavior
Postal code is not a five digit US ZIP	A clear refusal that states the US-only coverage. No outside call of any kind
ZIP code is well formed but not found	A clear "ZIP code not found" message. No call to Gemini
The weather API is slow or down	Use a **recent cached copy** if one exists and say how old it is. Otherwise a clear "weather data is unavailable" message. Never invent values
A field is missing for one period (for example UV)	Show **"not available"** for that item with the reason. The rest of the report still works. The advice must not rely on the missing number
Gemini returns an error, a quota message or an unusable answer	Show the **facts**, and give **rule-based advice** written in your own code, clearly labeled as such. Count the failure
The daily Gemini cap is reached	The same fallback as above, and a log line
The email service rejects or delays the message	Tell the user honestly. Do not claim "sent" until the service accepted it
An instance is terminated	The ALB and Auto Scaling Group recover it. Because no state lives on the instance, nothing the user needs is lost
A request takes too long	Time limits on every outside call, shorter than the ALB's 60 second idle timeout

Targets (record the measured results in your report):

Measure	Target
New report (nothing cached)	Under 20 seconds
Report from cache	Under 3 seconds
Recovery after a terminated instance	Within the target you set in lab04 and lab06, measured by your chaos test

10. Cost and Budget

Item	Expectation
Your stack (ALB, instances, public addresses)	About **$0.07 per hour** while it runs, as in lab06
Gemini	Free tier, within its limits
NWS and any free data source	No charge
DynamoDB, SSM standard parameters, SNS email	Effectively free at this scale
Project budget	**Under $2.00** in total for all build, test and demonstration time

At $0.07 per hour, $2.00 is about 28 hours of running time, so **build and test in short sessions and delete the stack every time you stop**. Adapt your `budget.py` to this project: hours the stack ran, Gemini calls counted, and the cost of everything that is not free. Your submission includes its output, with real numbers.

11. Engineering Requirements

11.1 Code and deployment

- All **deployment code is Python**: the stack package, the boot script that starts the application on each instance, the create and delete scripts, `budget.py` and the infrastructure tests. No bash, no shell commands, no `.sh` files.
- The **application** is Node.js (extending lab06) or Python. If it is Node.js, it is still started by your Python boot script, and its settings still come from `.env` through your Python code.
- Start with `cp -r lab06 final_project`. Never edit lab06.
- Every create function is **idempotent** and every delete function tolerates things that are already gone, as in lab06.
- Names, ports, limits and models come from `.env`, not from the code.
- The Gemini model name is a setting, because model names change.
- Small functions with docstrings. Pure functions for anything you can test without AWS (validation, aggregation, categories, prompt building, answer checking, policy documents, boot script generation).

11.2 Tests

Layer	What it must cover
Unit tests (no network), application logic	Every validation rule and its boundaries, including a table of **non-US postal codes that must be refused** (Canadian, UK, four and six digit numbers, letters, empty, very long) and ZIP codes that must be accepted. Period selection across time zones and daylight saving time changes. Aggregation. Rain and Snow thresholds on and around each boundary. Checking of Gemini's answer (good, malformed, wrong types, too long, hostile text). Escaping of anything shown on the page. The rule-based fallback advice. Policy documents contain no wildcards
Unit tests with moto	Tables, IAM, the full create and delete of the stack, idempotency, and the order of creation. The daily cap rule, including two requests racing
API contract tests	Responses **recorded from the real services** (saved as fixtures) are fed to your parsing code. This proves your code reads what the services really send. At least one test per source, including a response with a missing field
Email tests	The state machine for consent, the "only to the requester" rule, and the limits
Integration tests	Against the deployed stack: a real report for a real ZIP and date, a cached repeat, bad input, the health of both target groups, the app port reachable only through the ALB, the roles allowed and denied what they should be
Chaos tests	Terminate an instance and measure recovery. Force the Gemini cap and show the fallback. Make a data source unavailable and show the degraded report

Which language the tests use. Tests of the **application's own logic** are written in the application's language with its standard test runner. Tests of the **infrastructure and deployment**, and black-box checks of the deployed application over HTTP, are written in **Python with pytest**. If your application is Node.js, the rows below marked "application logic" run in Node.js and the rest in pytest.

`pytest -m "not integration and not chaos"` and your application's own test command must both pass with no AWS credentials and no network access.

11.3 What you may not do

- Put a number in the report that did not come from a data source or from your code's own calculation.
- Let Gemini's text reach the page without being checked and escaped.
- Store or log the email address or gender beyond what Section 7.4 allows.
- Leave the stack running after you finish a work session.

---

12. Decisions You Must Record

Your architecture document includes a **decision table**. Each row names a decision, the options you considered, what you chose, and why. At least these:

#	Decision
1	One tier, two tiers, or more, and what each owns
1a	Application language: extend the lab06 Node.js apps or write the application in Python, and why
2	Which API supplies each of the ten outputs, and what you do about gaps
3	How ZIP codes become coordinates, and how unknown ZIP codes are handled
4	Definition of Morning and Night, and how time zones and daylight saving time are handled
5	Aggregation of each measured item
6	Thresholds for Heavy, Mild and No, and their source
7	What is cached, for how long, with what key, and why
8	How Gemini is called (model, answer format, limits), and how its answer is checked
9	The rule-based fallback advice
10	SNS or SES, and the consent and privacy design
11	Per-tier IAM permissions
12	Abuse controls and their limits
13	How the daily cap and the budget are enforced
14	How you handle gender in the interface, the prompt and the logs

13. Deliverables

#	Deliverable	Contents
1	**Architecture document**	Diagram, component table, the two request flows, the data classification (Section 4.4), the decision table (Section 12), and a threat table
2	**Source map and API report**	The completed table from Section 5.1 with real example responses, the gaps you found, and your justification for every extra source
3	**Report definition**	Your answers to every item in Section 6
4	**Code**	The application, the Python stack, the Python boot script, scripts to create and delete everything, and the tests
5	**Test evidence**	The pytest summary, the integration and chaos results, and the measured response times and recovery time
6	**Cost report**	The output of your adapted `budget.py`, with the hours the stack really ran
7	**Live demonstration**	Build the stack from nothing with your create script, produce a report, send the email, break something on purpose, and delete everything
8	**Reflection**	One page: what you would change at ten times the traffic, and what you would change before putting this into real use

14. Suggested Evaluation Weights

Area	Weight
Architecture document and decision quality	20%
Data source investigation and report definition	15%
Application correctness (all ten outputs, both periods, all four inputs, email)	15%
Security and privacy	15%
Reliability and failure behavior	10%
Deployment code and idempotency (Python only)	10%
Tests	10%
Cost report, demonstration and reflection	5%

15. Common Pitfalls

- **Trusting the first API you find.** Check that it supplies every field for every day, not just today.
- **Letting the model do arithmetic or invent data.** Facts come from APIs and your code.
- **Using server time.** Morning at the user's location is not Morning in `us-east-2`.
- **One shared SNS topic with no filtering.** Everyone receives everyone's report.
- **Treating a missing value as zero.** Zero wind and "no wind data" are different things.
- **Forgetting the unhappy paths.** Every outside call fails sometimes, and the demonstration will show it.
- **Leaving the stack running.** An idle ALB costs the same as a busy one.

16. Key Terms

Term	Meaning
Normalized facts	The ten values per period, in fixed units and a fixed shape, no matter which API supplied them
Source map	The table linking each output to its API, field, evidence and gap handling
Derived category	A label (such as Heavy) that your code computes from measured values using published thresholds
Fallback advice	Advice written in your own code, used when Gemini cannot be used
Consent	The recipient's confirmation that they agree to receive mail from your topic or sender
Daily cap	A limit on Gemini calls per day, enforced by an atomic write in DynamoDB
Contract test	A test that feeds real recorded API responses to your parsing code
Idempotent	Running a create or delete step twice has the same result as running it once